Don't wait for the right time.

Create it.

trailer.
↑
movie.

→ Why linked-linked ?

→ Insertion in linked list
  → At start
  → At end
  → At $K^{th}$ index

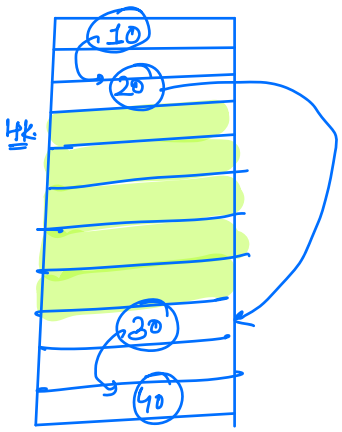
→ Deletion in linked list (#idea)
  → At start
  → At end
  → At $K^{th}$ index.

## Arrays.

int arr[5]

1 int → 4 B
5 int → 4×5 → 20 B.

{ T.C to access any random element of the array → O(1) }

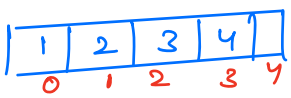



int arr[4] → {10, 20, 30, 40}

⇓

Since the contiguous space for 4 integers is not available.

X Not allowed.



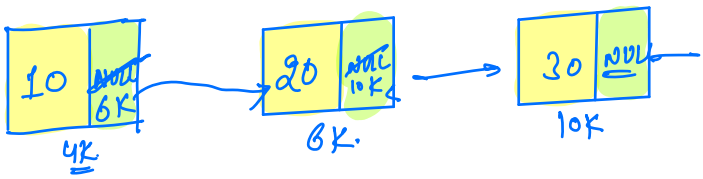

## Arrays.

| | At start. | At end. | Kth index |
|---|---|---|---|
| Insertion | O(N) | O(1) | O(N) |
| Deletion | O(N) (shifting) | O(1) | O(N) |

[structure → val, address of next element]

## LinkedList.

```
class Node {
    int val;
    Node next;
    Node(int x) {
        val = x
        next = null
    }
}
```



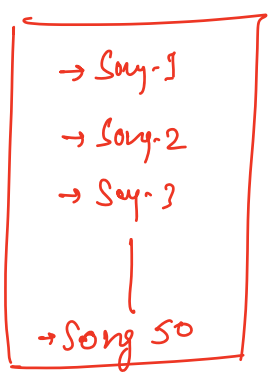

```
Node a = new Node(10);
a.next = new Node(20);
a.next.next = new Node(30);
a.next.next.next = new Node(40);
```

---

```
Node a = new Node(10);
Node b = new Node(20);
a.next = b;
Node c = new Node(30);
b.next = c;
```

* **Music Player**

  * Favorite.




* **Search engines**

ip → [iphone, ipad, ipad air, iphone covers]

D.S → inverted index

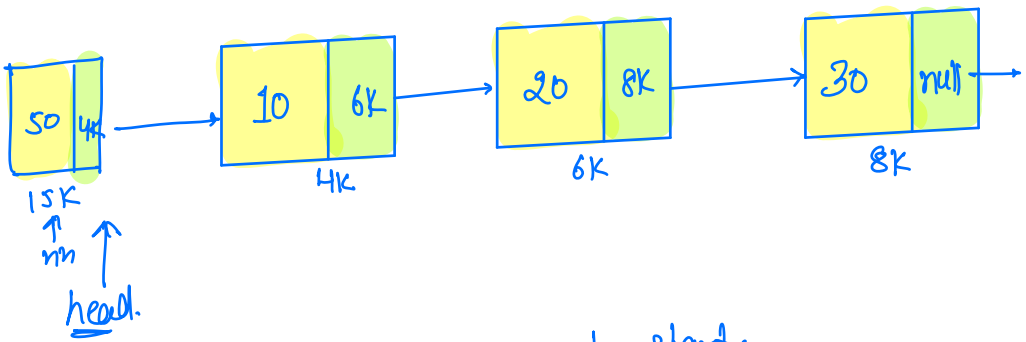
→ Appendix.
→ Glossary.

galaxy → [_, _, _, _, _]

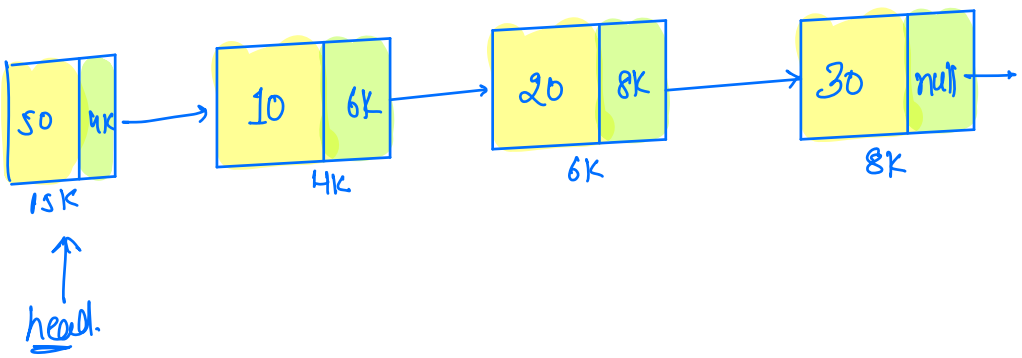
# Insertion At Start.

head.
address of first Node.

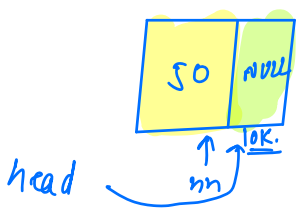
head.val
head.next.




→ insert 50 at start.



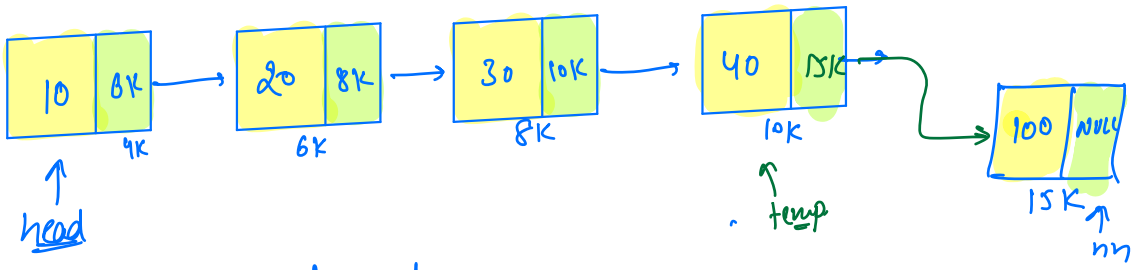

```
Node insertAtStart( Node head (4K), val)
{
    Node nn = new Node(val);
    nn.next = head;
    head = nn;
    return head;
}
```



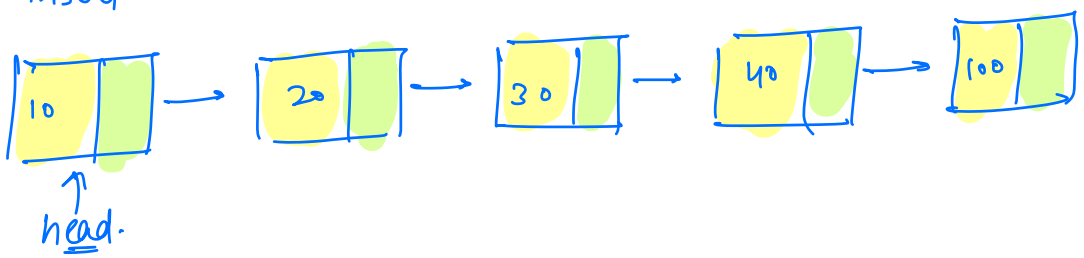

{ T.C → O(1)
S.C → O(1) }

```
Node a = new Node(10);
Node b = insertAtStart(a, 20);
Node c = insertAtStart(b, 30);
Node d = insertAtStart(c, 40);
```

# Insertion At End.




→ insert 100 at end.




```
Node insertAtEnd ( Node head, int val){
    Node nn = new Node (val);
    if ( head == NULL) { head = nn }
    else { Node temp = head;
        while ( temp.next != NULL){
            temp = temp.next
        }
        temp.next = nn;
    }
    return head;
}
```

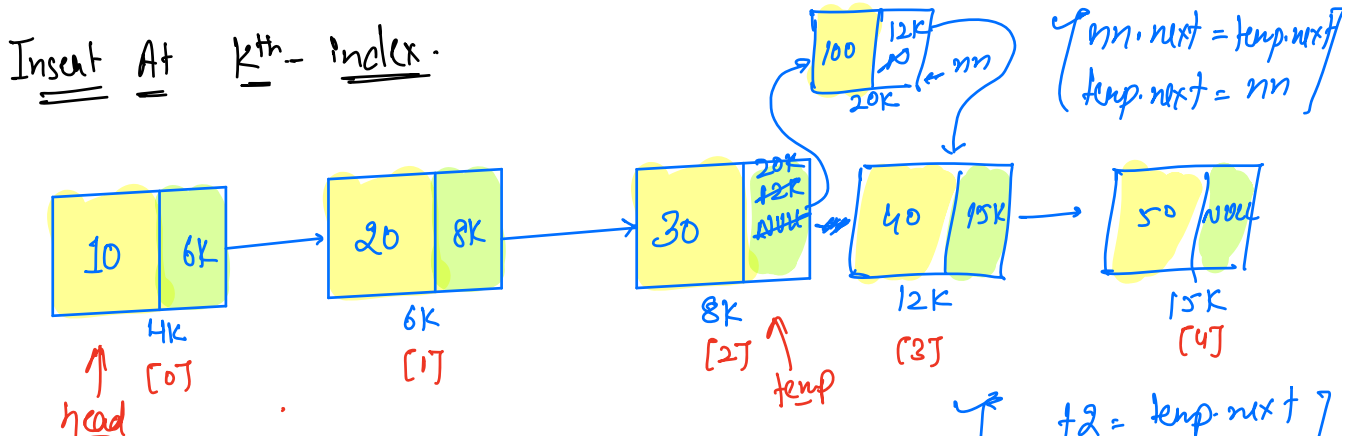

null.next

NULL Pointer Exception

{ T.C → O(N)
S.C → O(1) }

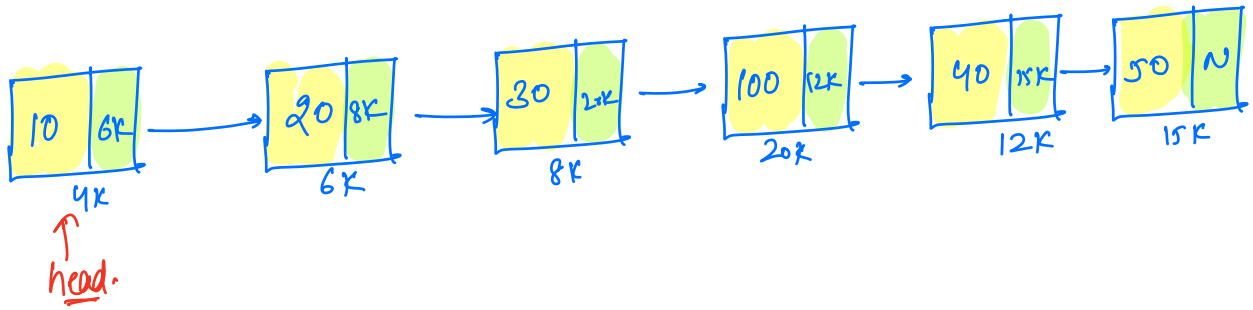
→ By using tail pointer → T.C → O(1)

# Insert At $K^{th}$ - index.




→ insert 100 at 3 index.

```
t2 = temp.next
temp.next = nn
nn.next = t2;
```



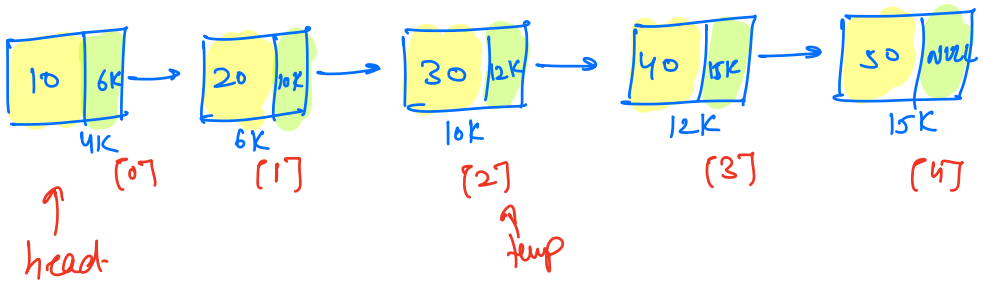

```
Node insertAtK ( head, val, K) {
    Node nn = new Node (val);
    if (K == 0) { insertAtStart (head, val); }
    else {  Node temp = head
            // update temp K-1 times
            for ( i = 1 ; i <= K-1 ; i++) {
                temp = temp.next
            }
            nn.next = temp.next;
            temp.next = nn;
            return head;
    }
}
```

T.C → O(N)
S.C → O(1)

K → 0 to N.

K → 0
K → N

# Deletion




## ① Delete At start

head = head.next

## ② Delet At End.

→ traverse upto second last node.

→ temp.next = NULL

## ③ Delete At $K^{th}$ index.

→ traverse upto $K-1^{th}$ idx.

temp.next = temp.next.next

(code - # todo)

[LL Trees.] → 1st, 2nd.

Node.
↓
head.

LinkedList<Int> l = new LinkedList();

LinkedList {
    Node head;
    Node tail;
    int size;
    getAt( );
}

→ {Do more & more questions.}